ReelSome Pro

Software Requirements Specification

Version 4

09 April 2026

Group 15

Aden Moon, Amani Paul, & Jaylan Luong

Prepared for
CS 250 - Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Spring 2026

Revision History

Date	Description	Author	Comments
19 Feb 2026	Wrote 1.2-1.3, 2.2-2.5, 3.3a, 3.3.3, 3.4a	Aden Moon	Also worked on formatting of the SRS document.
19 Feb 2026	Worked on External Interface Requirements and found possible APIs online that our Ticketing System can use.	Jaylan Luong	Currently unsure if all the APIs will be used for the Ticketing System but included anyway. Will narrow decisions as a group later.
26 Feb 2026	Implemented Use Cases	Amani Paul	Forgot to include diagrams showcasing use case scenarios
05 Mar 2026	Completed UML diagram and UML description	Aden Moon	Made corrections to points marked down from version 1 and worked on UML diagrams and the descriptions.
5 Mar 2026	Completed SWA Diagram and SWA Description	Jaylan Luong	Pretty sure that the SWA diagram holds all major functions of our ticketing system. Included APIs as External resources as a part of the diagram in a similar format to the SWA template on the modules in Canvas.
5 Mar 2026	Completed task responsibilities and timeline; added GitHub repo; updated section titles for new additions	Amani Paul	Unsure of how team responsibilities need to be specified; decided to include general sections with most importance
19 Mar 2026	Added test cases TC004, TC005, and TC006 that account for User.UserLogin(), integration tests for Cart_TheaterDatabase and Ticket_SendGrid respectively.Added some parts	Jaylan Luong	Tests fulfill the requirements for SDS Test Plan as a part of the rubric.

	of the Integration test plan write-up.		
19 Mar 2026	Completed test cases TC001, TC002, TC003, TC009, and TC010 that implement testing of units (functions) and systems (pages).	Amani Paul	Tested units and systems with appropriate priority levels for later fixes.
19 Mar 2026	Completed test cases TC007, TC008 that implement tests for timeRemaining in Cart and FoodNames in Food. Completed System information of Test Plan.	Aden Moon	Tests successful with appropriate steps and pre-requisites given.
9 Apr 2026	Wrote Data Management Strategy, discussing choice to implement SQL and other design choices that are reflected in the Software Architecture Diagram.	Amani Paul	Agreed upon how to implement the Database Management System, but still open to alternatives if current form is ineffective.
09 April 2026	Contributed to writing diagram modifications and discussion tradeoff.	Aden Moon	Discussed with team members about the diagram, tradeoffs, and alternatives.
09 April 2026	Created and Updated Software Architectural Diagram with MySQL and reformated databases	Jaylan Luong	Discussed the necessary modifications needed to update our previous diagram with bidirectional arrows to fit our current data management strategy.

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
Aden Moon	Aden Moon	Designer	09 April 2026
Jaylan Luong	Jaylan Luong	Designer	09 Apr 2026
Amani Paul	Amani Paul	Designer	9 Apr 2026

Table of Contents

REVISION HISTORY 2
DOCUMENT APPROVAL 3
1. INTRODUCTION 5
1.1 PURPOSE 5
1.2 SCOPE 5
1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS 5
1.4 REFERENCES 5
1.5 OVERVIEW 6
2. GENERAL DESCRIPTION 6
2.1 PRODUCT PERSPECTIVE 6
2.2 PRODUCT FUNCTIONS 6
2.3 USER CHARACTERISTICS....6
2.4 GENERAL CONSTRAINTS 6
2.5 ASSUMPTIONS AND DEPENDENCIES 7
3. SPECIFIC REQUIREMENTS 7
3.1 EXTERNAL INTERFACE REQUIREMENTS....7
3.1.1 User Interfaces 7
3.1.2 Hardware Interfaces 7
3.1.3 Software Interfaces....7
3.1.4 Communications Interfaces....8
3.2 FUNCTIONAL REQUIREMENTS 8
3.2.1 Ticket Checkout 8
3.3 USE CASES 8
3.3.1 Use Case #1....9
3.3.2 Use Case #2....10
3.3.3 Use Case #3....10
3.4 CLASSES / OBJECTS 11
3.4.1 Class Descriptions....11
3.5 NON-FUNCTIONAL REQUIREMENTS 12
3.5.1 Performance 12
3.5.2 Reliability 12
3.5.3 Availability 13
3.5.4 Security....13
3.5.5 Maintainability 13
3.5.6 Portability....13
4. SOFTWARE DESIGN SPECIFICATION....14
5. DEVELOPMENT PLAN & TIMELINE....16
6. TESTING....16
6.1 Test Plan....16
6.2 Test Case....18
6.2 Test Case....19
6.2 Test Case....20
6.2 Test Case....21
7 DATA MANAGEMENT....22
7.1.1 Software Architecture Diagram....22
7.1.1 Software Architecture Diagram....23

7.1.2 Updated Modifications.....23
7.2 Data Management Strategy.....23
7.3 Tradeoff Discussion.....24

1.1 Purpose:

The purpose of this SRS is to provide all the features and requirements for a movie ticketing system software that will be used by movie theaters across the country to sell their tickets digitally. The intended audience of this SRS is for software engineers that will be designing our software based on our specifications and for movie theater companies that will use our system to sell their tickets.

1.2 Scope:

ReelSome Pro is a mobile and web browser-friendly software that allows customers to find movies, purchase tickets, select seats, and order food/beverages which they will receive via email with a QR code. It provides movie theater staff functions for managing movies, showtimes, pricing, seating, and food through the software.

1.3 Definitions, Acronyms, Abbreviations:

- QR code: Digital ticket that is scannable on a mobile device.
- SRS: Software Requirements Specification.
- API: Application Programming Interface.
- SMTP: Simple Mail Transfer Protocol.
- SWA: Software Architecture.
- ACID: Atomicity, Consistency, Isolation, Durability

1.4 References:

- Cloudflare. (2025). What is https? *Cloudflare*.

  https://www.cloudflare.com/learning/ssl/what-is-https/

- Institute Of Electrical And Electronics Engineers. (1998). *IEEE recommended practice for software requirements specifications.* Ieee.

- Krüger, N. (2023, January 17). *How to write a software requirements specification (SRS document) | perforce software*. Perforce Software.

  https://www.perforce.com/blog/alm/how-write-software-requirements-specification-srs-document.

1.5 Overview:

The SRS will contain product context, its uses, and constraints. The detailed requirements will be given for functionality, interfaces, use cases, classes/objects, and non-functional requirements.

2. General Description

2.1 Product Perspective:

An online movie ticketing system provides and stores the following information:

- Movie Details:
  - Includes the movie names, images, dates, movie length, and show times, as well as the theater they are located in and seating availability.
- Customer Information:
  - If the customer has an account, their login and payment information are saved. Along with their past purchases and current number of reward points saved.
  - If the customer doesn't have an account, their email is saved during guest checkout, where their ticket ID is saved as a barcode/QR code.
- Ticket Information:
  - Includes the customer's name, movie name, time, theater, and seat.

2.2 Product Functions:

- **Customer:** Search movies and showtimes, view seating options, view pricing, purchase tickets, food, and beverages, access ticketing QR code, view order history, create an account, and request a refund.
- **Staff:** Create movies and showtimes, configure seating arrangements, configure pricing, update promotions/discounts, view sales, and view movie attendance.

2.3 User Characteristics

Users of the movie ticketing system should be able to view movies across different dates and times on a smartphone or website.

2.4 General Constraints

ReelSome Pro must work on modern browsers (Chrome/Safari/Firefox, etc.) without issue as well as on apps. The software will work properly on mobile devices without crashing. It must work with a third party company for secure transactions at checkout. Software will have security for users who create accounts and save credit card information.

2.5 Assumptions & Dependencies

Assume customers have a mobile device with internet access. Customers have a valid e-mail address to receive ticketing information. Customers have a charged device to show QR code in person at the theater. Third party payment company works properly for customer checkout. The movie theater must provide accurate movie showtimes and seating arrangements as well as updating seats/tickets already purchased. Movie theaters have access to QR code scanning equipment. Movie theaters will identify if the customer fits the description for their discounted tickets (adult, child, or senior) at the movie theater.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

- Home page
- Current/popular movies right now
- Movie details (length, actors, director, etc)
- Showtime selection by date and time
- Seat selection with open and taken indicators
- Cart to checkout
- Login for users with an account
- Order history in account
- Discounts/promotions
- Third party secure checkout
- Refund request
- Order confirmation
- QR code ticket
- Error handling for taken seats

3.1.2 Hardware Interfaces

- QR code scanner for tickets

3.1.3 Software Interfaces

- Paypal (Tracking payments)
    - Allows users to pay for tickets on the website.
- Sendgrid (Email API)
    - Send confirmation emails for purchased tickets.
- Google Maps API

    - Find nearby theaters with available tickets.
    - Sends users to Google Maps for directions to the theater.

3.1.4 Communication Interfaces

- HTTPS
    - Sends data between web browser and website.
    - Data transfer is encrypted for user's privacy.
- Rest APIs
    - Used to communicate between frontend and backend.
    - Users can login to their email, pay for tickets, etc.
- Simple Mail Transfer Protocol (SMTP)
    - Allows us to send emails to the customer.

3.2 Functional Requirements

3.2.1 Ticket Checkout

3.2.1.1 Introduction

This function holds the seats that the user selects for 2 minutes in their cart. If they do not purchase tickets within the 2-minute timer, these tickets will be available for users to buy in real time.

3.2.1.2 Inputs

The user selects on-screen seats labeled by their row and column numbers.

3.2.1.3 Processing

The website holds these seats for two minutes for the user and indicates to other users in real time that those ticket checkout holder seats are unavailable. If the tickets aren't bought within the two-minute timer, the tickets will be available for others, and the user will need to refresh the page to be able to put the tickets in their cart.

3.2.1.4 Outputs

If the user purchases tickets, our website will indicate to users that those seats are unavailable. The purchaser receives a confirmation email of the ticket with the seat number, movie name, and movie date.

3.2.1.5 Error Handling

- If a user's card gets declined when purchasing the tickets (insufficient funds), the user will be prompted with a message that their card was declined. Then the seats will be marked as available again.
- If a user tries to pick the same seat as another user had picked, the user will be prompted with a message that the seat was already taken.
- If the user does pay for their ticket within two minutes after adding their tickets in the cart, the cart will be cleared and the seats will be available again.
- If someone enters the wrong login information for their email, an error message will be prompted that their login information was wrong. If the user gets their login information wrong 5 times, the login will be timed out for 30 minutes to avoid mass logins.

3.3 Use Cases

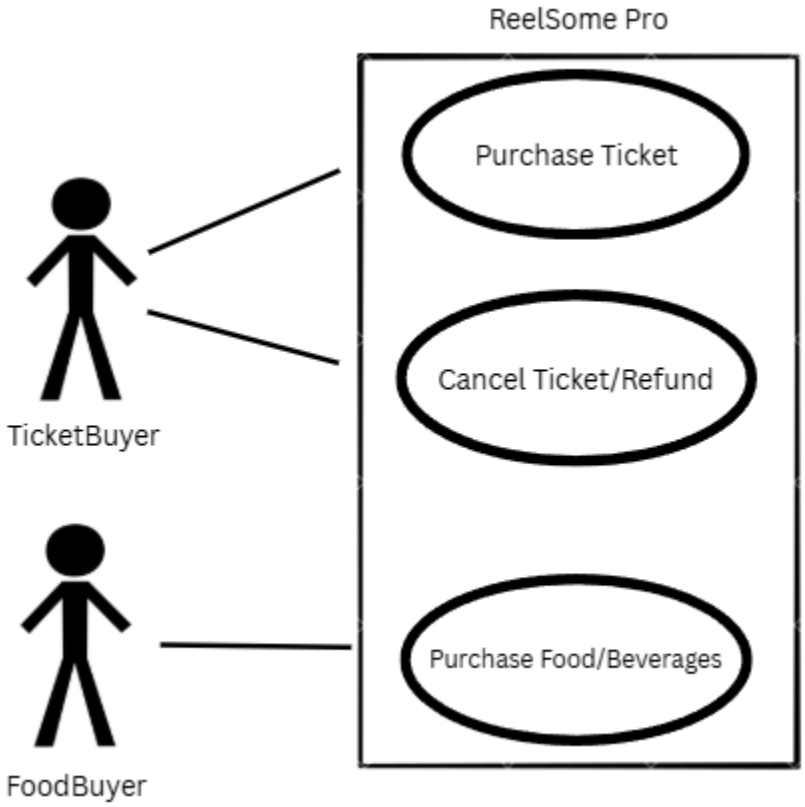


Figure 3.3: Use Cases

3.3.1 Use Case #1

Name: Purchase Ticket
Actor(s): TicketBuyer
Flow of events:

1. TicketBuyer connects to ReelSome Pro Server.
2. Server checks if TicketBuyer is logged in to an account already.
3. TicketBuyer selects a movie and date from a list of available screenings.
4. TicketBuyer chooses an available time and screening type from the chosen movie.
5. TicketBuyer then chooses a limited number of seats to be added to their cart, within a given time frame.
6. If TicketBuyer has an account and is logged in, they checkout with their saved payment method. Otherwise, they check out as a guest and add their email address and a payment method.
7. A confirmation email is sent to TicketBuyer, consisting of a QR Code for each purchased ticket.
8. If a Ticket Buyer has an account, their QR Code is also saved directly within their account.

3.3.2 Use Case #2

Name: Purchase Food/Beverages

Actor(s): FoodBuyer

Flow of events:

1. FoodBuyer connects to ReelSome Pro Server.
2. Server checks if FoodBuyer is logged in to an account already.
3. FoodBuyer selects a theater from a list of nearby locations.
4. FoodBuyer views a list of available food and beverage items, picking specified modifications and adding chosen items to their cart.
5. If FoodBuyer has an account and is logged in, they checkout with their saved payment method. Otherwise, they check out as a guest and add their email address and a payment method.
6. A confirmation email is sent confirming the purchased items.

3.3.3 Use Case #3

Name: Cancel Ticket/Request Refund

Actor(s): TicketBuyer

Flow of events:

1. TicketBuyer connects to ReelSome Pro server.
2. TicketBuyer selects previously purchased ticket from purchase history.
3. Server displays the ticket details (movie, date, time, seat number) and shows if cancellation is allowed.
4. Server checks to make sure the ticket hasn't already been used.
5. If ticket has not been used and is in allowable window for refund, the server processes the refund back to the original payment method.
6. Server will update the seat to be marked open.

7. A confirmation email is sent to the TicketBuyer showing proof of cancellation and refund.

3.4 Classes / Objects

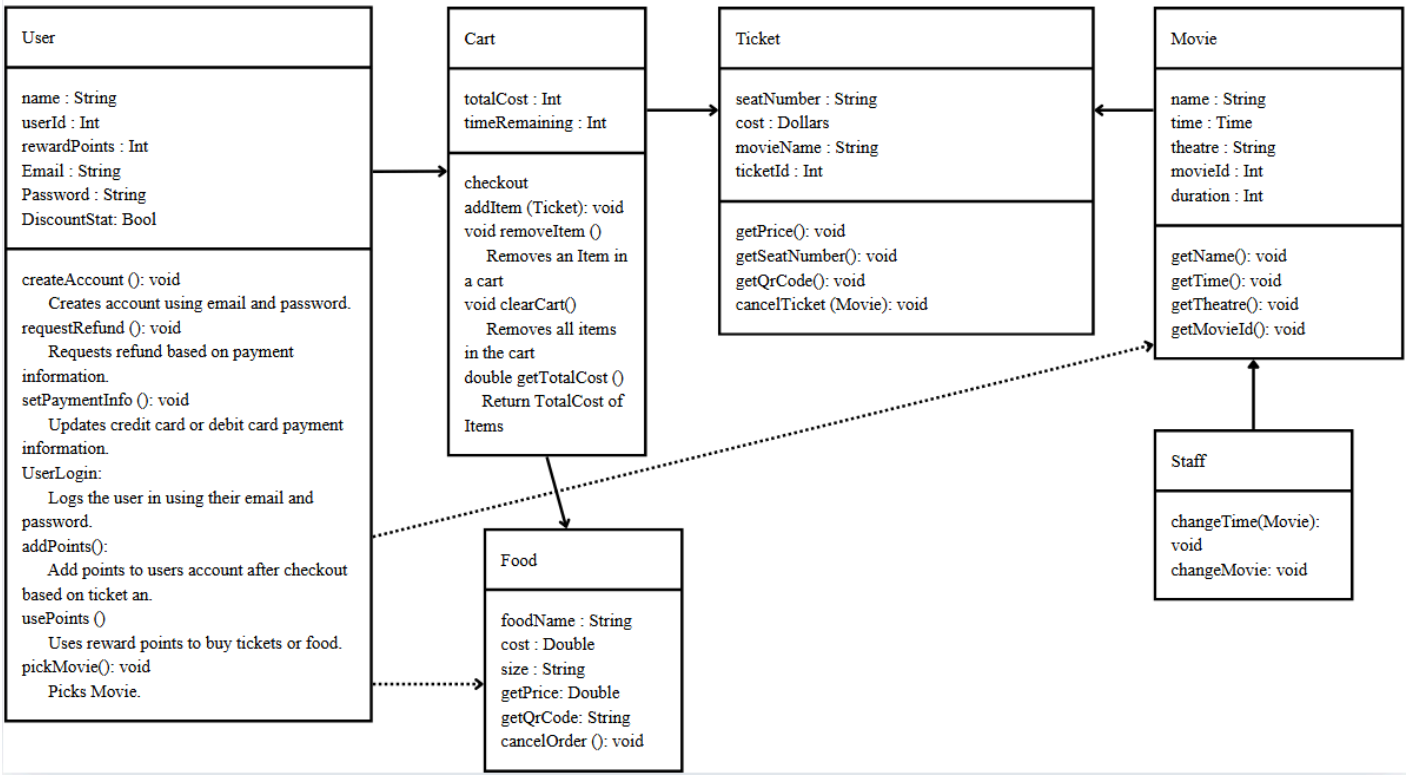


Figure 3.4: Classes / Objects

3.4.1 Class Descriptions:

The ReelSome Pro software consists of a few core classes that represent the users, purchases, and theater information. The following classes define the primary objects used in the system's framework.

User: Represents a customer using ReelSome Pro software to create an account, store account information, use reward points, request refunds, and pick movies.

- This class contains attributes such as name, userId, rewardPoints, Email, Password, and DiscountStat to store customer information.
- User class uses operations createAccount, requestRefund, setPaymentInfo, UserLogin, addPoints, usePoints, and pickMovie to allow the user to interact with the software.

Cart: Temporarily stores selected movie tickets, seats, and food items before checkout.

- The Cart class maintains attributes totalCost and timeRemaining to keep track of the cart value and seat time holder.
- Operations for this class include checkout, addItem, removeItem, clearCart, and getTotalCost to manage items in the cart.

Food: Shows food/beverage/candy items available for purchase and their costs.

- The attributes for the food class include foodName, cost, and size which store information for each item purchased.
- The Food class uses operations getPrice, getQrCode, and cancelOrder to get pricing information, generate the QR code for pick up, and cancel food orders if need be.

Ticket: Represents a purchased ticket containing the movie details, price, seat number, and unique ticket ID with QR code.

- The attributes used for the Ticket class include seatNumber, cost, movieName, and ticketId which are used to identify each ticket and store the details.
- The operations used are getPrice, getSeatNumber, getQrCode, and cancelTicket allow the software to get ticket information, generate a QR code, and cancel the ticket if needed.
-

Movie: Stores the movie information such as title, showtime, theatre number, and duration.

- This class utilizes attributes name, time, theatre, movieId, and duration for describing the movie details for its scheduled showing.
- The Movie class includes operations getName, getTime, getTheatre, and getMovieId which allow the software to retrieve information about the movie and its screening.

Staff: Represents the movie theatre staff responsible for managing movies, showtimes, seat information, and updating system data.

- The Staff class contains operations changeTime and changeMovie which update showtimes and modify movie information.

3.5 Non-Functional Requirements

3.5.1 Performance

The server will run and be readily available through web and app interfaces with high connection speeds and minuscule delays between function access and page changes. There will also be server expansions in case of events that induce high traffic on the server, to prevent reduced speeds and access to important features.

3.5.2 Reliability

ReelSome Pro Server should be consistent with the ability to perform its functions and services. In case of loss in network connectivity or the end of the connection with the server by user, then the server should save the last instance of the cart.

3.5.3 Availability

With a stable internet connection and update application, the server should be readily available at all times, unless in the case of a planned, timed maintenance (of which should be conducted during the least busy user traffic).

3.5.4 Security

The server will be protected from malicious attacks with the appropriate protective design philosophies and implementation of antivirus software. User info will be held in a protected database and efficiently encoded. To protect against bot activity, users will be required to solve a captcha.

3.5.5 Maintainability

Routine checks will be performed to make sure the server is properly running and the application is performing its intended functions effectively. As previously stated, these will take place during the times of least user traffic to ensure the software is available at the most important times.

3.5.6 Portability

The software would be able to be used on portable devices such as smartphones and laptops, with easy access to the user's account and information through simple log-in.

4. Software Design Specification

SWA:

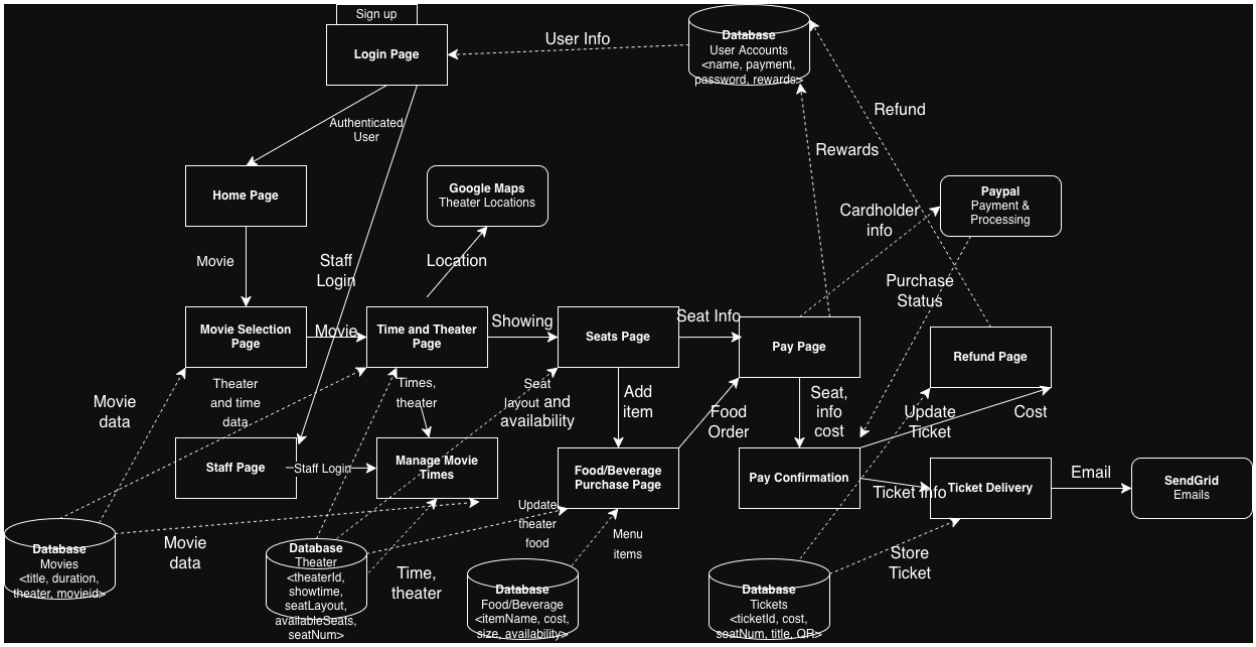


Figure 4.1 SWA

SWA Description:

- Login Page: This page communicates with a User Accounts Database that is used to authenticate the user for their login. Consumers will then be sent to the Home Page while Staff members are sent to the Staff Page.
- Home Page: The main page of the Reel Some Pro that navigates towards the movie selection page.
- Movie Selection Page: This page shows current available movies through retrieving movie data from the Movie Database. Movie data consists of the title, duration, theater, and movieId. Users can select a movie and will be prompted to the Time and Theater Page.
- Time and Theater Page: This page shows users the time and theaters that are airing that movie using the movie id from the movie database and the showtime and theaterId from the Theater Database. A Google Maps API is used for users to be able to locate theaters nearby that ReelSome Pro sells tickets for. After the user makes a selection, they are prompted to the Seats Page for the theater page.
- Seats Page: Users in this page can select their seats based on the seat layout and the available seats based on the theater Id that they chose from the Theater Database. Users will be able to choose a type of seat based on age range that will

be sent to the Pay Page. Users can then either add food/beverages and be sent to the Food/Beverage Purchase Page, or they can be sent other Pay Page.

- Food/Beverage Purchase Page: Users can add items to their total purchase that is communicated with the items they choose that is communicated with the Food/Beverage Database. The data sent are the itemName, cost, and availability of the food. This page also uses the theaterId from the theater database to display the correct foods based on the specific theater. After selections, user will be sent to the Pay Page.
- Pay Page: Collects all the information acquired based on the number of seats chosen and seat type from the Seats Page and the items selected from the Food/Beverage Purchase Page. This page uses the User Accounts Database to retrieve the necessary payment information and rewards that can be used for their total cost. A Paypal API is used to handle the user's payments using the information retrieved from the User Accounts Database. The Users will then be sent to the Pay Confirmation Page
- Pay Confirmation Page: Users will be able to see their order confirmation and the total cost of their tickets and items. This information will be sent to the Ticket Delivery. Users will also be able to refund their tickets by
- Ticket Delivery: Generates a QR code with the seat number and title of the film that is stored in the ticket in the Ticket Database. A confirmation email is sent to the user through the SendGrid.
- Refund Page: Users are able to refund their ticket using the payment information from the User Accounts Database. The ticket is then updated in the Tickets database to make the seat available again.
- Staff Page: Landing page for staff after login and is sent to manage movie times page
- Manage Movie Times: Allows staff to update movie, theater, and food/beverages information on their pages from their respective databases.
- External services: Paypal handles and processes all payments. Google Maps is used to provide theater locations and directions for users to the theaters. SendGrid handles emailing to users their tickets.

5. Development Plan & Timeline

- Team member responsibilities
  - Jaylan Luong - SWA Diagram and Description, External Interface Requirements, Overview
  - Aden Moon - UML Diagram and Class Description, General Description, Functional Requirements
  - Amani Paul - Use Cases, Non-Functional Requirements, Table of Contents, Tasks, GitHub Repository

- Timeline
  - 2/19: External Interface Requirements, Overview, General Description, Functional Requirements, Table of Contents
  - 2/26: Use Cases, Non-Functional Requirements
  - 3/5 : UML Diagram and Class Description, GitHub Repository, Tasks
  - 3/19 : Test Plan and Test Cases
- Repository
  - https://github.com/AmaniSDSU/CS-250-SRS

6. TESTING

6.1 Test Plan

- System
  - The ReelSome Pro software needs to work seamlessly throughout the entire software as a whole. The functionality to select a movie, date, and time is important but doesn't do anything if the user is unable to checkout in the cart page or a payment cannot go through successfully. As well as the functionality of a confirmation sent to the user via email with the movie details and QR code to present to staff at the theatre. Testing multi step functionality is vital for the success of the software. The system test plan will focus on end-to-end verification of the major workflows in the software. This includes ticket purchasing, seat reservations, payment processing, QR code generation, food ordering, and refund handling.
- Integration
  - Confirm that interfaces among integrated units are behaving properly and have appropriate access, such as users not having access to Staff methods, and Staff and Tickets being able to modify movie objects. Can staff members effectively modify movie showtimes and listings? Are tickets able to return movie seats to the database as available in the event of a customer refund? In addition, users should be able to perform these

actions readily and consistently. As mentioned above, it's important for customers to have the ability to refund their tickets (when applicable) and those tickets quickly become available for purchase. For security purposes, it is also crucial that customers don't have access to any integrated units that affect Movie objects. Databases also need to be working in conjunction with our pages and we see them being tested in TC005 and TC006. Users need to reliability transmit and receive accurate and valid data from the database when purchasing and receiving their tickets. If the user purchases their ticket as a guest, will they receive the right confirmation email? If two users add the same ticket to their cart what will happen? These are test cases that need to be accounted for the optimal user experience.

- Unit
  - Verify unit functionality with various test cases, specifically methods of the Cart, Staff, User, and Ticket classes, such as TC002, TC003, TC004, and TC010. Relevant units are Cart.clearCart(), Staff.changeTime(Movie), User.userLogin(), and User.userPoints(). Testing these classes will help us determine if their functions are performing their designated tasks and are meeting the system requirements. Are they accessing the right variables or returning the correct data types? Then we will see if they are meeting user expectations, such as customers being able to empty their cart and use their reward points, as well as staff having the ability to change movie showtimes. Regarding later fixes, each unit has a given priority for the urgency of their respective issues, with customer and staff login, cart, and purchase methods having the highest priority.

6.2. Test Cases

TestCaseId	Component	Priority	Description/ Test Summary	Pre-requisites	Test Steps	Expected Result	Actual Result	Status	Test Executed By
TC001	Purchase History Page	P0	Verify that a user can properly refund an unused ticket within a reasonable time	User is logged into system	1. Go to purchase history page 2. Select eligible ticket for refund 3. Customer receives refund confirmation email	User is refunded for ticket and the specific seating becomes available again for purchase	User is refunded, however the ticket does not become available again for purchase is remains un buyable	Fail	Amani Paul

TC002	Cart.clearCart()	P1	Verify that the user can empty their cart of any ticket and food items before purchase	Cart contains food or ticket items	1. Go to cart 2. Verify cart contains items 3. Select "empty cart" feature	User's cart should be emptied of any items and total cost is reverted back to 0.	The total cost of the cart is shown as 0 and no items are present	Pass	Amani Paul
TC003	Staff.changeTime (Movie)	P2	Confirm that staff can properly change movie showing times when necessary	User is logged in with proper credentials and movie time has not already passed	1. Access Manage Movie Times page 2. Select movie and change its time 3. Verify that movie time has changed	When the user checks current movie showtimes, the accessed movie's time should be different than prior to staff access	The movie time is still listed as the previous showtime, with no change shown for both staff and customers	Fail	Amani Paul
TC004	User.UserLogin()	P0	Verify that the user can log in	User already has an account within the system	1. Navigate to the Login Page 2. Enter in valid login credentials 3. Click login button	User successfully logs in and is redirected to the Home Page	User is successfully logged in and is redirected to the Home Page	Pass	Jaylan Luong
TC005	Cart_TheaterDatabase	P0	Verifies that when users add seats to their cart, their seats are marked as unavailable in the Theater Database	Movie and its available seats exist in the Theater Database and that the user is on the Seats Page.	1. Navigate to the Seat Page 2. Select a open seat 3. User 1 adds the seat to the cart 4. User 2 adds the same seat to their cart	User 2 sees that the seat is marked unavailable and User 1 has their seat in their cart within the TheaterDatabse.	User 2 was able to add the same seat to their cart and duplicate seats are marked to be taken by both users	Fail	Jaylan Luong
TC006	Ticket_SendGrid	P1	Verifies that a ticket purchase confirmation email with a QR code was emailed to the user	User has a valid email and has paid for their ticket with a valid payment method.	1. Finish paying for the ticket 2. As a guest, enter in a valid email address when prompted to send a	User successfully finds their ticket within their provided email with a QR Code and the correct movie	User's ticket confirmation email is sent to the wrong email address and it contains the wrong information of the ticket	Fail	Jaylan Luong

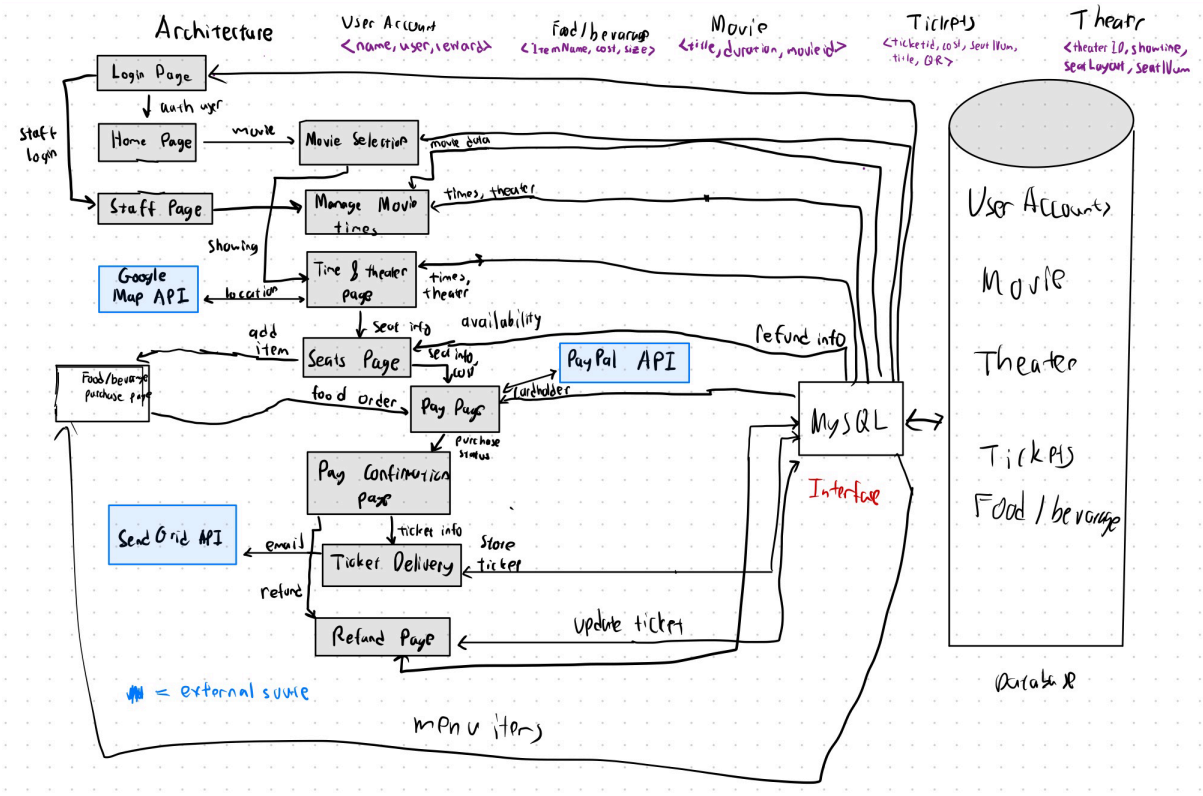
					confirmation email 3. Check email for the confirmation for the ticket 4. Verify tht the QR code is there and the ticket details are correct	details from the TheaterDatabse.	that was bought.		
TC007	FoodName	P1	Verify that a user can add food/beverage items to cart and purchase them.	Food/beverage menu items loaded in the system.	1. Navigate to food/beverage page. 2. Select food and beverage item. 3. Add items to cart. 4. Complete checkout.	Food items are added to cart, payment is processed, order confirmation sent to email.	Selected food and beverage items were successfully added to cart. Payment accepted, checkout successful. Order confirmation displayed and emailed to user.	Pass	Aden Moon
TC008	Cart.timeRemaining	P0	Verify that held seats are released after 2 minute cart-timer expires without checkout.	Seat added to cart.	1. Select seat and add to cart. 2. Wait more than 2 minutes without checking out. 3. Refresh seat page.	Seat held in cart expires, cart is cleared, and seat becomes available again.	The seat remained reserved in the cart for the 2 minute hold period. After timer expired without a checkout, the cart was cleared and seat made available again for purchase.	Pass	Aden Moon

TC009	Staff Page	P1	Confirm that the staff page can be accessed with proper login credentials and provides users with access to relevant functions	User has staff login information	1. Got to log in page and enter userId and password.	User is given access to the Staff page	User proceeds to the Staff page, where they have access to functions such as changeTime(Movie) and changeMovie	Pass	Amani Paul
TC010	User.usePoints()	P10	Validate the ability for customers to consume their points to purchase food and tickets	User has available points and enough to redeem the cheapest item	1. Proceed through Movie selection page 2. Continue through time and theatre page then seat selection 3. Once at Food/Beverage Purchase Page use points to add desired items to cart	When the user redeems their points, they proceed with their checkout and the appropriate number of points are removed from their account	After redeeming points, the number of available points listed on the user's account remains the same	Fail	Amani Paul

Repository: https://github.com/AmaniSDSU/CS-250-SRS

7. Data Management

7.1.1 Software Architecture Diagram



7.1.2 Updated Modifications

Compared to our original design in section 4, this iteration focuses more on the data management and system interactions. The initial SWA diagram showed how users can navigate through the ReelSome Pro software pages but didn't fully show how information was stored, accessed, and updated. The updated version here shows a dedicated database for the different types of data in the software, making the system more organized. It clearly shows how the application interacts with external APIs like SendGrid, PayPal, and Google Maps. Additionally, the updated diagram reflects more realistic behavior by showing how actions like the seat selection, ticket purchasing, and ticket refunds involve backend processes and database updates. These modifications make the system more complete with how a modern web application should be structured.

7.2 Data Management Strategy

As our software systems contain sensitive information and must be consistent with constantly changing data, our strategy is to apply a SQL database. Using a database approach for our data management, we'll use a Database Management System as our interface that will store the data and keep it organized and coordinate updates. This allows for easier data modifications and the avoidance of data redundancy. The most beneficial properties of a SQL database that will help maintain our systems are its relational properties, following ACID properties (Atomicity, Consistency, Isolation, Durability), run-time flexibility, and good fitting for high transaction-based applications. By using a database for each object type, our data will be better organized and be an efficient use of storage space. However, using a DBMS has its own disadvantages, such as being more complex to manage, difficulty in recovering from failures, as all files and programs share the same space, and lastly, increased costs to purchase and operate. While NoSQL databases are an option, many of their properties aren't well-suited for the type of movie ticketing system we are implementing. They lack powerful queries, are mainly non-relational, not fit for heavy transactions or run-time flexibility, and aren't reliable in providing consistent data. As our system is planned to be readily available for all customers to purchase/refund tickets and have consistent and accurate data to avoid seats not showing as available when they in fact are, or multiple customers purchasing the same tickets.

7.3 Tradeoff Discussion

When designing the data management strategy, several alternatives were considered before choosing a SQL based database. The primary goal of the ReelSome Pro system is to handle real-time transactions like ticket purchases, seat selection, and ticket refunds while maintaining accurate data across all users using the software. The decision to use a SQL database over a NoSQL database was a major decision. A NoSQL database is flexible and good for handling large amounts of unorganized data, however the strict consistency, especially with transactions, is much better with a SQL database. Making sure every transaction is 100% accurate is more in line with the service our application will provide. In a movie ticketing system, consistency is very important, for example if multiple users are trying to select a seat at the same time. Without strict transaction control, there could be incorrect seat availability or duplicate seat selections. Overall, the SQL database is a better fit because of its secure transactions through ACID technology.

Another decision involved whether to use a single centralized database or the alternative to use multiple separate databases. Using multiple databases allows different parts of the system like user accounts, tickets, and food orders to be managed independently. However, this approach increases the system's complexity and would likely increase development time. Using a single relational database keeps all related data in one place which is beneficial because it is easier to maintain consistency and manage transactions. This reduces the risk of data conflicts and is also faster for development time.

Overall, the chosen design by our team prioritizes consistency, reliability, and simplicity over the alternative of scalability and flexibility. The tradeoff is important for the ReelSome Pro system because it is a transactional system. Accurate and real-time data updates are the most important for how we need the system to perform.

Repository: https://github.com/AmaniSDSU/CS-250-SRS